

# Experiment 9

Student Name: Aman Raj Ojha  
Branch: CSE - AIML  
Semester: 4  
Subject Name: DBMS

UID: 24BAI70520  
Section/Group: 24AIT\_KRG G1  
Date of Performance: 1/4/26  
Subject Code: 24CSH-298

## Aim

To understand and implement database triggers in PostgreSQL to automate data validation and computational logic, ensuring data integrity by enforcing business rules during DML operations.

## Software Requirements

- Database Management System:
  - Oracle Database Express Edition (Oracle XE)
  - PostgreSQL Database
- Database Administration Tool / Client Tool:
  - Oracle SQL Developer (for Oracle XE)
  - pgAdmin (for PostgreSQL)

## Objectives

- To create a trigger function that performs automatic calculations on row-level data.
- To define a BEFORE INSERT trigger that intercepts data entry for validation.
- To implement custom exception handling using the RAISE EXCEPTION command.
- To verify trigger behaviour by testing both valid and invalid data scenarios within a transaction.

## Practical/Experiment Steps

- Schema Provisioning: Established the employee table with dedicated columns for base data (hours and rate) and a calculated column for the final payable amount.
- Trigger Function Development: Authored a PL/pgSQL function designed to calculate the product of working hours and hourly rates dynamically.
- Automated Validation Logic: Embedded a conditional check within the function to prevent the insertion of records where the calculated amount exceeds a predefined limit.
- Trigger Binding: Configured a row-level trigger to execute the function specifically BEFORE any INSERT operation occurs on the table.
- Operational Stress Testing: Utilized anonymous DO blocks to simulate data entry and verify that the system correctly identifies and blocks prohibited transactions.

## Procedure

- Accessed the PostgreSQL console and created the employee table structure.
- Defined the CALCULATE\_AMOUNT() function using the NEW keyword to access and modify incoming row data before it is written to the disk.
- Integrated a mathematical formula to populate the total\_payable\_amount field automatically.
- Created the CAL\_PAYABLE\_AMOUNT trigger, ensuring it was set to FOR EACH ROW to evaluate every individual record.
- Executed a successful INSERT test case where the calculated result was below 25,000 to confirm normal operations.
- Ran a SELECT query to verify that the trigger correctly computed and stored the total amount despite it not being provided in the INSERT statement.
- Attempted an invalid INSERT with values resulting in an amount over 25,000 to trigger the custom exception.
- Implemented exception handling in the test blocks to capture and display the SQLERRM (error message) generated by the trigger.
- Analysed the final table state to confirm that the invalid record was successfully blocked by the trigger logic.

## Input/Output Analysis

### SQL Input Queries

CREATE TABLE employee (

```
emp_id INT PRIMARY KEY,  
emp_name VARCHAR(50),  
working_hours INT,  
perhour_salary NUMERIC,  
total_payable_amount NUMERIC  
);
```

## Output

| Data Output                              | Messages | Notifications |
|------------------------------------------|----------|---------------|
| CREATE TABLE                             |          |               |
| Query returned successfully in 266 msec. |          |               |

## SQL Input Queries

```
CREATE OR REPLACE FUNCTION CALCULATE_AMOUNT()  
RETURNS TRIGGER  
AS  
$$  
BEGIN  
    NEW.total_payable_amount=NEW.perhour_salary*NEW.working_hours;  
    IF NEW.total_payable_amount>25000 THEN  
        RAISE EXCEPTION 'AMOUNT IS GREATER THAN 25000';  
    END IF;  
    RETURN NEW;  
END;  
$$  
LANGUAGE PLPGSQL;
```

## Output

| Data Output                              | Messages | Notifications |
|------------------------------------------|----------|---------------|
| CREATE FUNCTION                          |          |               |
| Query returned successfully in 120 msec. |          |               |

### SQL Input Queries

```
CREATE OR REPLACE TRIGGER CAL_PAYABLE_AMOUNT
BEFORE INSERT
ON EMPLOYEE
FOR EACH ROW
EXECUTE FUNCTION CALCULATE_AMOUNT();
```

### Output

Data Output Messages Notifications

```
CREATE TRIGGER
```

```
Query returned successfully in 137 msec.
```

### SQL Input Queries

```
DO
$$
BEGIN
    INSERT INTO EMPLOYEE(EMP_ID, EMP_NAME,
WORKING_HOURS, PERHOUR_SALARY) VALUES
    (1, 'AKASH', 10, 250);

    EXCEPTION
    WHEN OTHERS THEN
    RAISE NOTICE '%', SQLERRM;
END;
$$;

SELECT * FROM EMPLOYEE;
```

### Output

Data Output Messages Notifications

```
DO
```

```
Query returned successfully in 96 msec.
```

Data Output

Messages

Notifications

≡

+

📄

▼

📋

▼

🗑️

▼

🗄️

▼

⬇️

▼

📈

▼

SQL

|   | <div>emp_id</div> <div>[PK] integer</div> | <div>emp_name</div> <div>character varying (50)</div> | <div>working_hours</div> <div>integer</div> | <div>perhour_salary</div> <div>numeric</div> | <div>total_payable_amount</div> <div>numeric</div> |
|---|-------------------------------------------|-------------------------------------------------------|---------------------------------------------|----------------------------------------------|----------------------------------------------------|
| 1 | 1                                         | AKASH                                                 | 10                                          | 250                                          | 2500                                               |

## SQL Input Queries

DO

\$\$

BEGIN

```
INSERT INTO EMPLOYEE(EMP_ID, EMP_NAME,
WORKING_HOURS, PERHOUR_SALARY) VALUES
(1, 'AKASH', 10, 250000);
```

EXCEPTION

WHEN OTHERS THEN

RAISE NOTICE '%', SQLERRM;

END;

\$\$;

```
SELECT * FROM EMPLOYEE;
```

## Output

| Data Output | Messages                                                                                   | Notifications |
|-------------|--------------------------------------------------------------------------------------------|---------------|
|             | NOTICE: AMOUNT IS GREATER THAN 25000<br>DO<br><br>Query returned successfully in 145 msec. |               |

Data Output

Messages

Notifications

≡

+

📄

▼

📋

▼

🗑️

▼

🗄️

▼

⬇️

▼

📈

▼

SQL

|   | emp_id<br>[PK] integer | emp_name<br>character varying (50) | working_hours<br>integer | perhour_salary<br>numeric | total_payable_amount<br>numeric |
|---|------------------------|------------------------------------|--------------------------|---------------------------|---------------------------------|
| 1 | 1                      | AKASH                              | 10                       | 250                       | 2500                            |

## Learning Outcomes

- **Trigger Lifecycle Mastery:** Understanding the difference between statement-level and row-level triggers and when to use BEFORE vs AFTER events.
- **Automated Data Calculation:** Proficiency in using triggers to derive and populate column values, reducing the logic burden on the application layer.
- **Custom Constraint Enforcement:** Ability to implement complex business rules and validations that go beyond standard CHECK constraints.
- **Error Handling and Signaling:** Mastery of the RAISE EXCEPTION mechanism to communicate specific business logic failures to the user or application.